CASE STUDY REPORT
Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization

A Technical Case Study on Tuning a Fuzzy Signal Controller with a Genetic Algorithm



Submitted by

Mohammed Amaan Khan 25WU0101075

Mentor Name: Dr Bhargav Prajwal Pathri

Academic Year: 2025-2029

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my mentor, **Dr Bhargav Prajwal Pathri**, from the **School of Technology, Woxsen University**, for his guidance, continuous support and encouragement throughout the completion of this **case study** titled "**Intelligent Traffic Signal Control Using Fuzzy Logic and Optimization**." His feedback during Review 1 and Review 2 shaped the way the problem was framed and pushed me to test the controller properly instead of only showing that it runs.

I also thank the **School of Technology, Woxsen University**, for providing the resources and the academic environment that made this project possible, and the faculty of the Foundations of Artificial Intelligence course, whose lectures on intelligent agents, fuzzy inference and evolutionary optimization are the base on which this work stands.

I would like to acknowledge everyone who, directly or indirectly, helped me during this work, including my classmates who sat through early versions of the presentation and asked the questions I had not thought of.

Finally, I am grateful to my family and friends for their constant encouragement and motivation.

Submitted By

Mohammed Amaan Khan

25WU0101075

Mentor Signature

ABSTRACT

Most traffic signals in Indian cities still run on fixed timers. Every road gets the same green time in every cycle, whether twenty vehicles are waiting or none. A traffic police officer standing at the same junction does better, because the officer looks at the queues and judges them as short, medium or long before deciding how long to hold the green. This report presents a technical case study of a signal controller that reasons in the same way. The controller is a Mamdani fuzzy system with two inputs (the queue on the road that has green and the longest queue among the roads on red), one output (how many seconds to extend the green) and nine rules. Its membership functions are then tuned by a Genetic Algorithm so that the average waiting time of vehicles is as low as possible.

The study was carried out on a purpose-built simulator of an isolated four-way intersection written in Python with NumPy. Four controllers were compared on identical traffic: a 30 second fixed timer, the best fixed timer found by a sweep (20 seconds), the fuzzy controller with hand-set membership functions, and the fuzzy controller after tuning. Three demand patterns were used (light, heavy and unbalanced), and every reported figure is the mean of ten one-hour runs on traffic the Genetic Algorithm never saw during training. In total the study executed **11,302 one-hour simulation runs**.

The tuned fuzzy controller reduced average waiting time against the 30 second timer by 65% in light traffic, 50% in heavy traffic and 71% in unbalanced traffic, and it still waited 51%, 43% and 68% less than the best fixed timer. Tuning alone accounted for a further 11% to 35% over the hand-set fuzzy controller. The score on unseen traffic (19.8 s) matched the training score (19.8 s), so the tuned controller did not overfit. Throughput was within 1% for all four controllers, which shows that the gain is in delay and not in volume. The study also explains what the optimizer changed: the tuned controller asks for shorter extensions but asks far more often, which avoids the three second switching penalty while vehicles are still queued. A browser prototype that runs the same model live accompanies the report.

Keywords: traffic signal control, fuzzy logic, Mamdani inference, membership functions, Genetic Algorithm, intelligent agent, optimization, simulation.

INDEX

S. No.	Contents	Page No.
	Abstract	3
	List of Figures	7
	List of Tables	7
1	**Introduction**	9
1.1	Background	9
1.2	Problem Statement	9
1.3	Motivation and Societal Relevance	10
1.4	Objectives	10
1.5	Scope and Delimitations	10
1.6	Case Study Method	11
1.7	Organisation of the Report	11
2	**Literature Review**	12
2.1	Fixed-Time Signal Control	12
2.2	Fuzzy Sets and Fuzzy Control	12
2.3	Fuzzy Logic at Traffic Junctions	12
2.4	Adaptive Network Systems	13
2.5	Genetic Algorithms for Tuning Fuzzy Controllers	13
2.6	Summary of Existing Approaches	14
2.7	Research Gap	14
3	**System Requirements and Analysis**	15
3.1	Functional Requirements	15
3.2	Non-Functional Requirements	16
3.3	Hardware and Software Requirements	16
3.4	Technology Stack	17
3.5	Repository Structure	17
4	**System Design and Architecture**	18

S. No.	Contents	Page No.
4.1	Architectural Overview	18
4.2	The Signal as an Intelligent Agent	18
4.3	Intersection Model	19
4.4	Data Model	20
4.5	Key Design Decisions	20
5	**Methodology and Implementation**	21
5.1	Traffic Simulator	21
5.2	Fuzzy Controller	21
5.3	Genetic Algorithm	23
5.4	Metrics Computation	24
5.5	Interactive Prototype	25
6	**Experimental Setup**	27
6.1	Evaluation Criteria	27
6.2	Simulation Parameters	27
6.3	Traffic Scenarios	28
6.4	Controllers Compared	28
6.5	Training and Test Protocol	28
7	**Results and Analysis**	30
7.1	Choosing the Fixed-Time Baseline	30
7.2	Waiting Time	30
7.3	Full Results	31
7.4	Convergence of the Genetic Algorithm	32
7.5	What the Optimizer Changed	33
7.6	Decision Surfaces	34
7.7	Behaviour Over Time and Consistency	35
7.8	Cross-Check with the Browser Prototype	36
7.9	Summary Against Objectives	37
8	**Discussion**	38

S. No.	Contents	Page No.
8.1	Strengths	38
8.2	Limitations	38
8.3	Educational and Societal Impact	39
8.4	Lessons Learned	39
9	**Conclusion and Future Scope**	40
9.1	Conclusion	40
9.2	Future Scope	41
	References	42
	Appendix A: Fuzzy Engine and Controller	43
	Appendix B: Genetic Algorithm	44

LIST OF FIGURES

S. No.	Figure	Page No.
1	Figure 4.1: Layered architecture of the system	18
2	Figure 4.2: Intersection model with one queue per road and one green phase at a time	19
3	Figure 5.1: One simulated second of signal operation	21
4	Figure 5.2: Hand-set membership functions of the three fuzzy variables	22
5	Figure 5.3: The Genetic Algorithm used to tune the membership functions	23
6	Figure 5.4: Prototype during a run of the tuned controller in unbalanced traffic	25
7	Figure 5.5: Prototype benchmark panel and the membership functions in use	26
8	Figure 7.1: Mean wait for each fixed green time on the training traffic (best value highlighted)	30
9	Figure 7.2: Average waiting time per vehicle on unseen traffic (bars show the mean of 10 runs, whiskers one standard deviation)	30
10	Figure 7.3: Best and mean fitness of the population over 40 generations	32
11	Figure 7.4: Membership functions before (dashed grey) and after (solid orange) tuning	33
12	Figure 7.5: Green extension for every pair of inputs; the black line is the 4 s switch threshold	34
13	Figure 7.6: Queue on the busy road during one unseen hour of unbalanced traffic	35
14	Figure 7.7: Run-to-run spread of the average waiting time over the 10 test seeds	36

LIST OF TABLES

S. No.	Table	Page No.
1	Table 2.1: Comparison of existing approaches	14
2	Table 3.1: Functional requirements and where they are implemented	15
3	Table 3.2: Non-functional requirements	16

S. No.	Table	Page No.
4	Table 3.3: Hardware and software requirements	16
5	Table 3.4: Technology stack	17
6	Table 3.5: Repository contents	17
7	Table 4.1: PEAS description of the signal agent	19
8	Table 4.2: The five parameters of one fuzzy variable	20
9	Table 4.3: Design decisions and trade-offs	20
10	Table 5.1: Rule base (cell value is the green extension)	22
11	Table 5.2: Genetic Algorithm settings	24
12	Table 6.1: Evaluation criteria and evidence	27
13	Table 6.2: Simulator parameters	27
14	Table 6.3: Traffic scenarios	28
15	Table 6.4: Controllers compared	28
16	Table 6.5: Simulation runs in the study	29
17	Table 7.1: Results on unseen traffic (mean of 10 one-hour runs per scenario)	31
18	Table 7.2: Reduction in average waiting time	32
19	Table 7.3: Hand-set and tuned membership parameters	33
20	Table 7.4: Extension in seconds for sample inputs	35
21	Table 7.5: Average waiting time in seconds, Python experiment vs. browser prototype	36
22	Table 7.6: Results summarised against the objectives	37
23	Table 9.1: Prioritised roadmap	41

1. INTRODUCTION

1.1 Background

A signalised intersection is a shared resource. Only one group of movements can use the junction at a time, so the signal has to divide one hour of time between several roads. The simplest way to do this is a fixed-time plan: each road receives a preset green period in a fixed order, and the plan repeats all day. Webster's formula from 1958 [1] gives the best fixed cycle for a known, steady flow, and it is still the starting point of signal timing manuals [14].

The weakness of a fixed plan is that it cannot see the road. Demand changes with the hour, the weather and local events, and at any moment one approach may be crowded while another is empty. A fixed timer gives both the same green. The result is familiar to anyone who has waited at a red light while the crossing road stood empty: green time is wasted on one side and a queue grows on the other.

A human controller handles this naturally. The officer does not count vehicles exactly. The officer sees that one queue is “long” and another is “short” and acts on that judgement. Fuzzy logic, introduced by Zadeh [2], was designed to capture this kind of reasoning with vague categories, and Mamdani showed that a small set of linguistic rules can drive a real controller [9]. In the language of the Foundations of AI course, the signal becomes an intelligent agent: it perceives the queues, reasons with rules and acts on the lights [11].

1.2 Problem Statement

A fuzzy controller needs membership functions, the shapes that say how strongly a queue of, say, seven vehicles counts as Medium or Long. In most published fuzzy signal controllers these shapes are chosen by the designer. Nothing guarantees that the designer's guess is a good one, and there is no formula that connects the shapes to the waiting time, because the waiting time only appears after the traffic has been simulated. The problem addressed by the project is therefore:

> **Problem.** To design a traffic signal controller for a four-way intersection that decides green time from the live queue on each road using fuzzy logic, and to tune that controller with an optimization algorithm so that the average waiting time of vehicles is as low as possible.

1.3 Motivation and Societal Relevance

Waiting at signals has a direct cost. Engines idle, fuel burns, exhaust collects at the busiest points of a city, and ambulances and buses are held with everyone else. A controller that cuts waiting without any new road space is one of the cheapest improvements available to a city. Fuzzy control is attractive for this purpose because its rules can be read and questioned by a traffic engineer who is not a programmer. A rule such as "if the green road queue is Long and the waiting queue is Short, extend the green for a Long time" can be checked against common sense. That transparency matters for a public system.

The project also has an educational motive. It brings together three ideas taught separately in the course (agents, fuzzy inference and Genetic Algorithms) in one small system where each part has a visible job.

1.4 Objectives

The objectives of the project, as stated in the Review 1 presentation, are:

1. **Simulate** a four-way signalised intersection in Python with random vehicle arrivals on each road.
2. **Design** a Mamdani fuzzy controller that reads queue lengths and outputs how long to extend the green.
3. **Optimize** the membership functions of that controller with a Genetic Algorithm to minimise average waiting time.
4. **Compare** three kinds of controller on the same traffic: fixed-time, fuzzy, and fuzzy tuned by the Genetic Algorithm.
5. **Test** under light, heavy and unbalanced traffic to see where each controller wins or fails.

1.5 Scope and Delimitations

- The world is one isolated four-way intersection. Networks of connected junctions and signal coordination are not modelled.

- Each road is a single queue. Turning lanes, pedestrian phases and emergency vehicles are outside the scope.
- Traffic is simulated with random arrivals. No camera, sensor or field data is used.
- The phase order is fixed (North, East, South, West). The controller decides how long each green lasts, not which road goes next.
- The Genetic Algorithm tunes the membership functions only. The nine rules are kept as designed.

1.6 Case Study Method

The report treats the project as an engineering artefact and reports what was measured, including the parts that did not go as expected. Four sources of evidence were used:

- **The source code** of the simulator, the fuzzy engine and the Genetic Algorithm (one Python file of about 260 lines), which is described module by module in Chapter 5.
- **A controlled experiment** in which all controllers face the same seeded traffic, with separate seeds for training and testing (Chapter 6).
- **Analysis of the tuned controller**: its membership functions and its full decision table were compared with the hand-set version to explain where the improvement comes from (Chapter 7).
- **An independent re-implementation** of the model in JavaScript for the browser prototype, whose results were checked against the Python results.

1.7 Organisation of the Report

Chapter 2 reviews earlier work on signal control and identifies the gap. Chapter 3 states the requirements and the technology used. Chapter 4 describes the design of the system and Chapter 5 its implementation. Chapter 6 defines the experimental setup and Chapter 7 reports the results. Chapter 8 discusses strengths, limitations and impact, and Chapter 9 concludes with future scope.

2. LITERATURE REVIEW

2.1 Fixed-Time Signal Control

Webster [1] derived a formula for the cycle length and green split that minimise average delay at an isolated junction when the flow on each approach is known and steady. The method is simple, needs no detectors and is still widely used, but its assumption is also its limit: when the real flow differs from the design flow the plan is no longer the best one, and it cannot react to a sudden change. Signal timing practice therefore prepares several plans for different times of day [14], which helps with predictable peaks but not with minute-to-minute variation.

2.2 Fuzzy Sets and Fuzzy Control

Zadeh [2] proposed that an element can belong to a set to a degree between 0 and 1. A queue of seven vehicles can then be partly Medium and partly Long at the same time. Mamdani and Assilian [9] used this idea to build a controller from linguistic if-then rules: inputs are converted to membership degrees (fuzzification), the rules are evaluated with the minimum operator for AND, the rule outputs are combined with the maximum operator, and a single crisp value is recovered as the centroid of the combined shape (defuzzification). This procedure, now called Mamdani inference, is the one used in this project and is described in standard texts [12].

2.3 Fuzzy Logic at Traffic Junctions

Pappis and Mamdani [3] were the first to apply a fuzzy controller to a traffic junction. On a junction of two one-way roads they reported lower delay than a conventional vehicle-actuated controller. Trabia, Kaseko and Ande [6] later divided the decision into two fuzzy stages for an isolated intersection with turning traffic, and Niittymaki and Pursula [7] showed in simulation that fuzzy control can compete with traditional vehicle-actuated control. In all three studies the rule base and the membership functions were set by expert judgement and adjusted by hand for the tested site.

2.4 Adaptive Network Systems

Large adaptive systems take a different route. SCOOT [4] and SCATS [13] adjust cycle, split and offset across a city network from detector data in real time. They are effective and widely deployed, but they depend on detectors at every approach, a communication network and a central computer, which makes them costly to install and maintain. They are a different class of solution from a self-contained controller at a single junction.

2.5 Genetic Algorithms for Tuning Fuzzy Controllers

A Genetic Algorithm [8], [10] keeps a population of candidate solutions, scores each one, and builds the next population by selecting good candidates and combining and mutating them. It needs only the score of each candidate, not a gradient, which makes it suitable when the score comes out of a simulation. Karr [5] showed that a Genetic Algorithm can tune the membership functions of a fuzzy controller automatically and that the tuned controller can outperform the one designed by hand. His demonstrations were on general control problems and not on traffic. Particle Swarm Optimization [15] is a common alternative for the same task.

2.6 Summary of Existing Approaches

Table 2.1: *Comparison of existing approaches*

Work	Approach	Contribution	Limitation
Webster, 1958 [1]	Fixed-time formula	Best cycle length and green split from average flow	Assumes steady demand; cannot react to sudden changes
Pappis and Mamdani, 1977 [3]	Fuzzy controller	First fuzzy controller for a traffic junction; lower delay than vehicle-actuated control	Two one-way roads; rules and shapes written by hand
Hunt et al., 1981 [4]	SCOOT adaptive system	Adjusts timings online from detector data across a network	Needs detectors and central infrastructure; costly
Karr, 1991 [5]	Genetic Algorithm + fuzzy	Membership functions tuned automatically	Shown on general control problems, not traffic
Trabia et al., 1999 [6]	Two-stage fuzzy	Handles an isolated intersection with turning traffic	Membership functions set by expert judgement
Niittymaki and Pursula, 2000 [7]	Fuzzy vs. actuated	Simulation evidence that fuzzy control is competitive	Rule base tuned manually for the tested site
This project	Fuzzy + Genetic Algorithm on one simulator	Measures the gain of fuzzy control and of tuning separately, on unseen traffic	Simulated single junction; rules not optimized (Chapter 8)

2.7 Research Gap

Three points emerge from the review. First, **fuzzy logic fits the problem**: queue length is naturally vague and fuzzy sets handle that well. Second, **hand tuning is the weak point**: in most fuzzy signal papers the membership functions come from the designer's judgement. Third, **optimizers can do the tuning**, and they do not need a formula for the objective. What is harder to find is a clear side by side study, on one simple and reproducible simulator, of how much a fuzzy signal improves over a fixed timer and how much more is gained when its membership functions are tuned by a Genetic Algorithm instead of by hand, with the tuning and the testing done on different traffic. This project addresses that gap.

3. SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Functional Requirements

Table 3.1: *Functional requirements and where they are implemented*

ID	Requirement	Implemented in
FR-1	Generate random vehicle arrivals for four roads from given rates, reproducibly from a seed	`make_arrivals()`
FR-2	Simulate one hour of signal operation in one-second steps with discharge and lost time	`simulate()`
FR-3	Provide a fixed-time controller with a configurable green time	`FixedController`
FR-4	Provide a fuzzy controller that extends green from two queue readings	`FuzzyController`, `build_table()`
FR-5	Evaluate nine Mamdani rules and defuzzify with the centroid method	`memberships()`, `build_table()`
FR-6	Tune the 15 membership parameters with a Genetic Algorithm	`run_ga()`, `repair()`
FR-7	Find the best fixed green time as a stronger baseline	sweep in `main()`
FR-8	Report average wait, average queue, maximum queue and throughput	result dictionary of `simulate()`
FR-9	Test every controller on unseen seeds in three scenarios and save the results	`main()`, `results.json`
FR-10	Draw the comparison charts from the saved results	`make_charts.py`
FR-11	Show the model running live in a browser for demonstration	`index.html`, `sim.js`, `app.js`

3.2 Non-Functional Requirements

Table 3.2: *Non-functional requirements*

Category	Requirement	Status in current build
Reproducibility	Same seed gives the same traffic and the same result	Met; arrivals and the Genetic Algorithm are both seeded
Fairness	All controllers face identical traffic	Met; arrival arrays are generated once and reused
Performance	The full experiment finishes in minutes on a laptop	Met; about one minute after the lookup-table change (Section 4.5)
Validity	Results reported on traffic not used for tuning	Met; 3 training seeds and 10 separate test seeds per scenario
Interpretability	Rules and tuned shapes can be read by a person	Met; nine rules and 15 numbers, plotted in Chapter 7
Portability	Runs without special hardware or licensed software	Met; Python with NumPy only, prototype is static files
Realism	Model matches real junction behaviour	Partly met; single-lane queues and fixed phase order (Section 8.2)
Maintainability	Modular code with tests	Partly met; clear module split, no automated tests

3.3 Hardware and Software Requirements

Table 3.3: *Hardware and software requirements*

Item	Minimum / used
Hardware	Any laptop or desktop; the study was run on a personal Windows 11 laptop
Language	Python 3 (study used Python 3.13)
Libraries	NumPy for arrays and random arrivals; Matplotlib for charts
Prototype	Any current browser; plain HTML, CSS and JavaScript with no build step
Cross-check	Node.js to run the JavaScript simulator outside the browser

3.4 Technology Stack

Table 3.4: *Technology stack*

Layer	Technology	Role
Simulation	Python standard library	Signal loop, queues, metrics
Numerics	NumPy [17]	Poisson arrivals, vectorised fuzzy inference
Fuzzy engine	Custom NumPy code	Membership functions, Mamdani inference, centroid, lookup table
Optimizer	Custom Genetic Algorithm (`random` module)	Selection, crossover, mutation, repair
Charts	Matplotlib [18]	Bar charts, convergence curve, membership plots
Prototype	HTML Canvas and JavaScript	Live animation, metrics and benchmark in the browser
Data exchange	JSON	`results.json` feeds the charts and the prototype

3.5 Repository Structure

Table 3.5: *Repository contents*

Path	Responsibility
`traffic_fuzzy_ga.py`	Simulator, both controllers, fuzzy engine, Genetic Algorithm and the experiment runner
`make_charts.py`	Reads `results.json` and writes the charts into `charts/`
`results.json`	Saved output of one full experiment: metrics, GA history, sweep, tuned genes, queue traces
`sim.js`	JavaScript port of the simulator and fuzzy engine
`app.js`, `index.html`	Prototype interface: animation, live metrics, decision panel, benchmark
`data.js`	Tuned and hand-set genes exported from `results.json` for the browser
`docs/`, `README.md`	This report and the run instructions

4. SYSTEM DESIGN AND ARCHITECTURE

4.1 Architectural Overview

The system is organised in four layers (Figure 4.1). The **simulator** knows nothing about fuzzy logic: it moves vehicles through four queues and asks a controller object two questions, how long the first green should be and whether to extend it. The **controllers** answer those questions. The **fuzzy engine and the Genetic Algorithm** produce the lookup table that the fuzzy controller reads. The **experiment layer** builds the traffic, runs everything and saves the results.

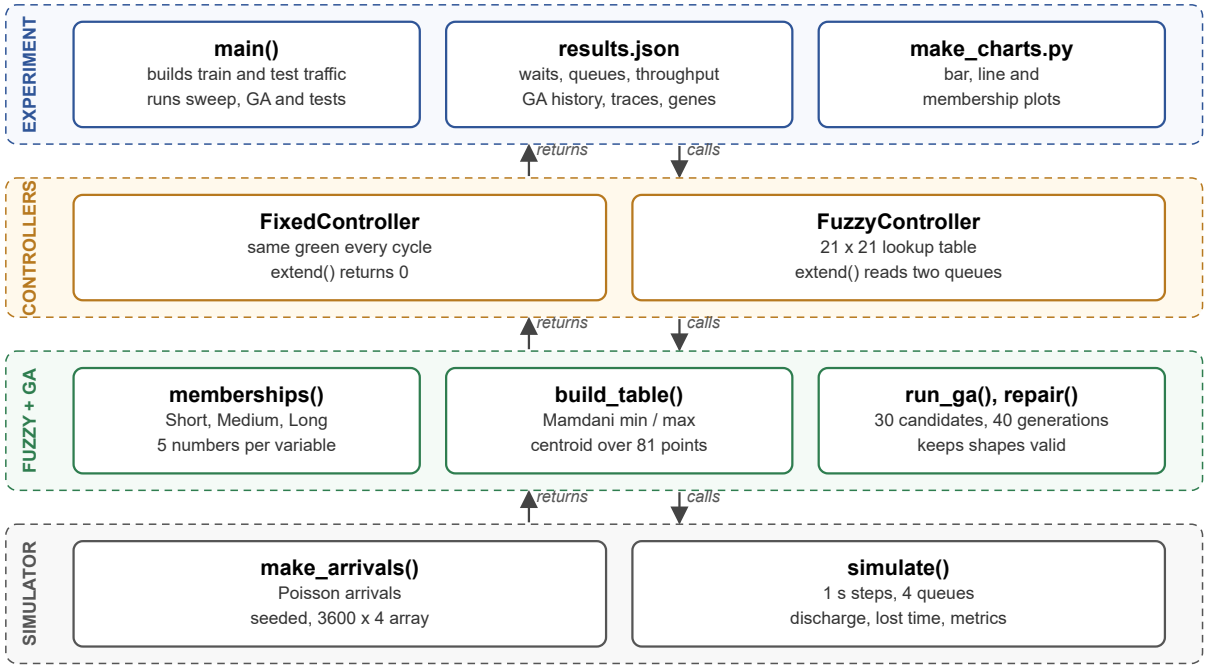


Figure 4.1: *Layered architecture of the system*

The most important design decision is that the simulator and the controller meet at a two-method interface. Because of it, a fixed timer and a fuzzy controller run in exactly the same loop on exactly the same arrivals, which is what makes the comparison fair, and a new controller can be added without touching the simulator.

4.2 The Signal as an Intelligent Agent

The controller is a simple reflex agent with a small amount of internal state (the time already spent in the current green). Table 4.1 describes its task environment in the PEAS form used in

the course [11].

Table 4.1: *PEAS description of the signal agent*

Element	In this project
Performance measure	Average waiting time per vehicle (main); average and maximum queue; throughput
Environment	Four-way intersection with random arrivals; partially observable, stochastic, sequential, dynamic
Actuators	The signal lights: extend the current green by a number of seconds, or switch to the next road
Sensors	Queue length on the road that has green; longest queue among the three roads on red

4.3 Intersection Model

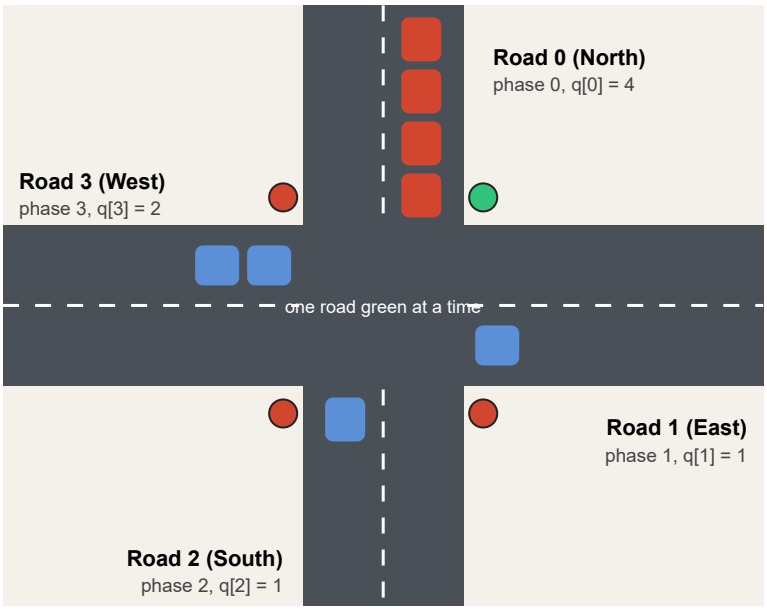


Figure 4.2: *Intersection model with one queue per road and one green phase at a time*

Each road is one queue. Vehicles arrive on each road independently in every second, the number being drawn from a Poisson distribution with that road's rate. While a road has green, one vehicle leaves every two seconds, which corresponds to a saturation flow of 1,800 vehicles per hour, a common planning value [14]. Whenever the signal changes road, three seconds are lost to the yellow and all-red interval and nobody moves. This lost time is the reason switching too often is costly, and it plays a large part in the results.

4.4 Data Model

A controller is described by 15 numbers, five for each of the three fuzzy variables (Table 4.2). The same 15 numbers are the chromosome of the Genetic Algorithm, so there is no separate encoding step.

Table 4.2: *The five parameters of one fuzzy variable*

Parameter	Meaning	Hand-set value (queues)	Hand-set value (extension)
short_end	Point where membership in Short falls to 0	8	8
medium_left	Point where Medium starts to rise	3	4
medium_peak	Point where Medium equals 1	10	10
medium_right	Point where Medium falls back to 0	17	16
long_start	Point where Long starts to rise (it reaches 1 at 20)	12	12

Every simulation run returns the same result dictionary: `avg_wait`, `avg_queue`, `max_queue`, `throughput` and, when requested, the second-by-second queue on one road. The experiment writes these, the history of the Genetic Algorithm and the tuned genes to `results.json`.

4.5 Key Design Decisions

Table 4.3: *Design decisions and trade-offs*

Decision	Rationale	Trade-off
Two-method controller interface	Any controller runs in the same loop on the same traffic	The controller sees only what the simulator passes to it
Precompute fuzzy inference into a 21 x 21 table	The Genetic Algorithm needs 11,070 one-hour runs; a table lookup per decision keeps this to about a minute	Inputs are rounded to whole vehicles and clipped at 20
Write the fuzzy engine in NumPy instead of using scikit-fuzzy	Full control over speed; all 441 table cells are computed in one vectorised pass	More code to write and check
Tune membership functions only	Keeps the rules readable and the search space small (15 genes)	A better rule base may exist and is not searched
Separate training and test seeds	Reported numbers measure generalisation, not memorisation	More simulation runs
Fixed phase order	Isolates the effect of green duration	Empty roads still receive the 6 s minimum green

5. METHODOLOGY AND IMPLEMENTATION

5.1 Traffic Simulator

Traffic for one run is a 3600 by 4 array of arrival counts produced by `make_arrivals(rates, seed)`. Generating the whole hour in advance means that two controllers given the same array face exactly the same vehicles at exactly the same seconds. The function `simulate(controller, arrivals)` then advances the junction one second at a time, as shown in Figure 5.1.

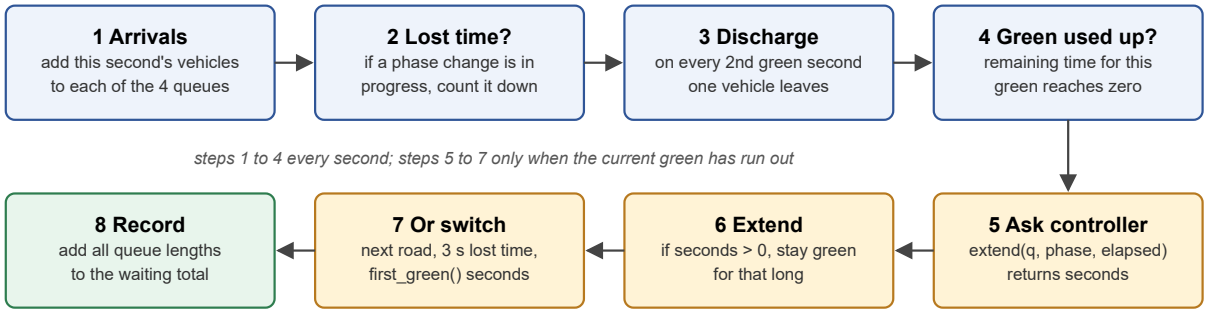


Figure 5.1: *One simulated second of signal operation*

The fixed controller returns its green time from `first_green()` and always answers 0 to `extend()`, so it switches as soon as its green is used. The fuzzy controller starts every phase with the minimum green of 6 seconds and then extends it in steps for as long as its rules justify, up to a maximum of 60 seconds.

5.2 Fuzzy Controller

5.2.1 Variables and membership functions

The controller has two inputs and one output, each on a scale of 0 to 20 with three fuzzy sets, Short, Medium and Long. Input 1 is the queue on the road that currently has green. Input 2 is the longest queue among the three roads on red. The output is the green extension in seconds. Short is a left shoulder, Medium a triangle and Long a right shoulder. Figure 5.2 shows the shapes chosen by hand before any tuning. They are the natural first guess: evenly spread and symmetric about the middle of the scale.

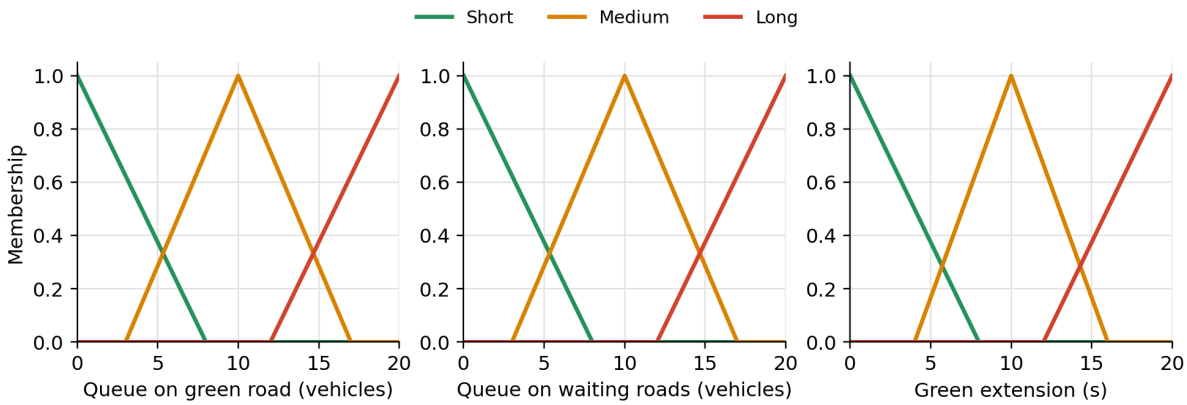


Figure 5.2: *Hand-set membership functions of the three fuzzy variables*

5.2.2 Rule base

The nine rules are given in Table 5.1. They express two ideas. A longer queue on the green road deserves a longer extension, and a longer queue on the waiting roads argues for a shorter one. When the green road queue is Short the green is never extended much, whatever the others are doing.

Table 5.1: *Rule base (cell value is the green extension)*

Green road queue	Waiting queue: Short	Waiting queue: Medium	Waiting queue: Long
Short	Short	Short	Short
Medium	Medium	Medium	Short
Long	Long	Medium	Medium

5.2.3 Inference and defuzzification

For a pair of crisp inputs, each rule fires with a strength equal to the minimum of its two input memberships. Rules that share an output set are combined with the maximum. Each output set is clipped at its firing strength, the clipped shapes are merged with the maximum, and the crisp extension is the centroid of the merged shape, computed over 81 sample points between 0 and 20 seconds.

> **Worked example.** Green road queue 12, waiting queue 2, hand-set shapes. Green queue: Medium = (17 − 12) / 7 = 0.71, Short and Long = 0. Waiting queue: Short = (8 − 2) / 8 = 0.75, Medium and Long = 0. Only the rule “Medium and Short gives Medium” fires, with strength min(0.71, 0.75) = 0.71. The Medium output triangle (4, 10, 16) clipped at 0.71 is symmetric about 10, so the centroid is **10.0 seconds**. The lookup table holds exactly this value in cell [12][2].

5.2.4 From extension to action

The controller turns the crisp value into an action with three guards. If the green road is empty it switches at once. If the green has already lasted 60 seconds it switches. If the computed extension is under 4 seconds it also switches, because an extension that short is not worth holding the other roads for. Otherwise the green continues for the computed number of seconds and the question is asked again when they run out.

5.2.5 Lookup table

Inference for all 21 by 21 integer input pairs is computed once per controller by `build_table()` using NumPy broadcasting. During simulation a decision is a single table read. This change was not in the Review 1 plan, which proposed the scikit-fuzzy library. It was made when it became clear that the Genetic Algorithm would need 11,070 one-hour runs: computing a centroid at every decision of every run would have been far too slow.

5.3 Genetic Algorithm

The Genetic Algorithm searches the 15 membership parameters while the rules stay fixed. Figure 5.3 shows one generation and Table 5.2 the settings.

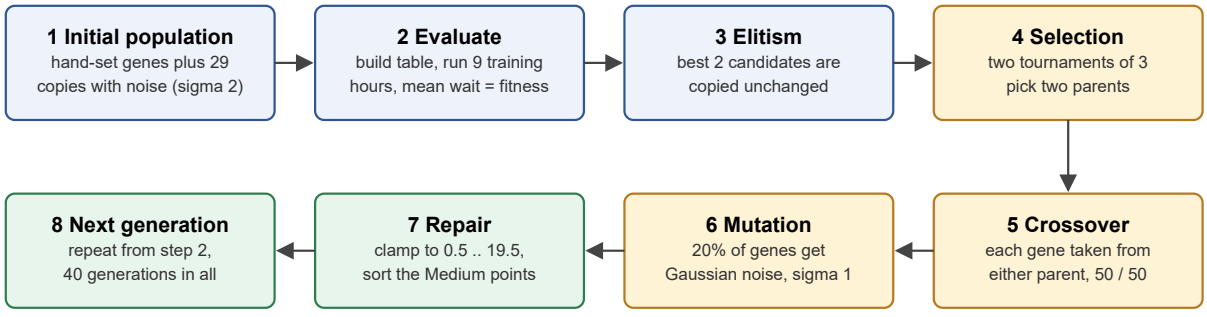


Figure 5.3: *The Genetic Algorithm used to tune the membership functions*

Table 5.2: *Genetic Algorithm settings*

Setting	Value	Reason
Chromosome	15 real numbers	Five per fuzzy variable (Table 4.2)
Population	30	Large enough for variety, small enough to evaluate quickly
Generations	40	The best score stopped improving noticeably after about 30
Selection	Tournament of 3	Simple, and does not need scaled fitness values
Elitism	Best 2 kept	The best candidate found is never lost
Crossover	Uniform, gene by gene	Parameters of different variables can be inherited independently
Mutation	20% of genes, Gaussian, sigma 1.0	Small moves around a good solution
Fitness	Mean wait over 3 scenarios x 3 seeds	One controller must suit all three kinds of traffic

The **repair step** is what keeps every candidate a valid fuzzy controller. After crossover and mutation a child may have its Medium points out of order or outside the scale. Repair clamps every gene into the range 0.5 to 19.5, sorts the three Medium points and forces a gap of at least 0.5 between them so that no division by zero can occur. The fitness uses the same training traffic for every candidate, so two candidates differ only because of their genes and not because one happened to receive easier traffic.

A Genetic Algorithm was chosen over a gradient method because the fitness is the output of a simulation with integer queues and a hard switch threshold. There is no expression to differentiate, and small changes in a gene often change nothing until a decision flips.

5.4 Metrics Computation

- **Average waiting time** is the sum of all queue lengths over all seconds, divided by the number of vehicles that arrived. Each queued vehicle adds one second of waiting for every second it stays in a queue, so the sum is the total vehicle-seconds of waiting. Vehicles still queued at the end of the hour are included.
- **Average queue** is the same sum divided by 3600: the mean number of vehicles waiting across all four roads.

- **Maximum queue** is the longest queue seen on any single road during the run.
- **Throughput** is the number of vehicles that left the junction in the hour.

5.5 Interactive Prototype

To demonstrate the system, the model was ported to JavaScript and placed behind a small web interface (Figure 5.4). The page animates the junction, lets the viewer switch between the four controllers and the three scenarios at any time, and shows the live waiting time, the queues and the reasoning behind the last decision of the fuzzy controller, including the firing strength of each output set. A benchmark button runs all four controllers for ten one-hour runs in the browser and draws the comparison (Figure 5.5).

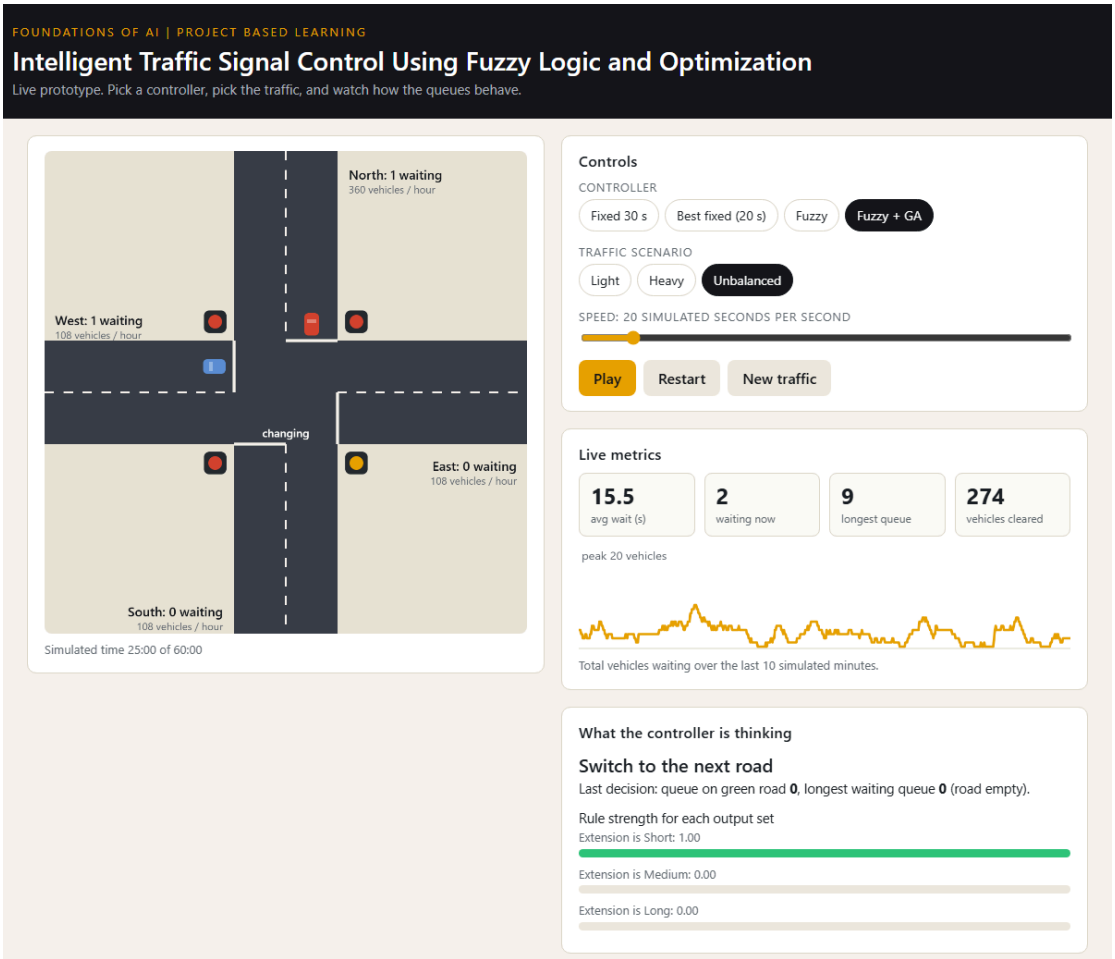


Figure 5.4: *Prototype during a run of the tuned controller in unbalanced traffic*

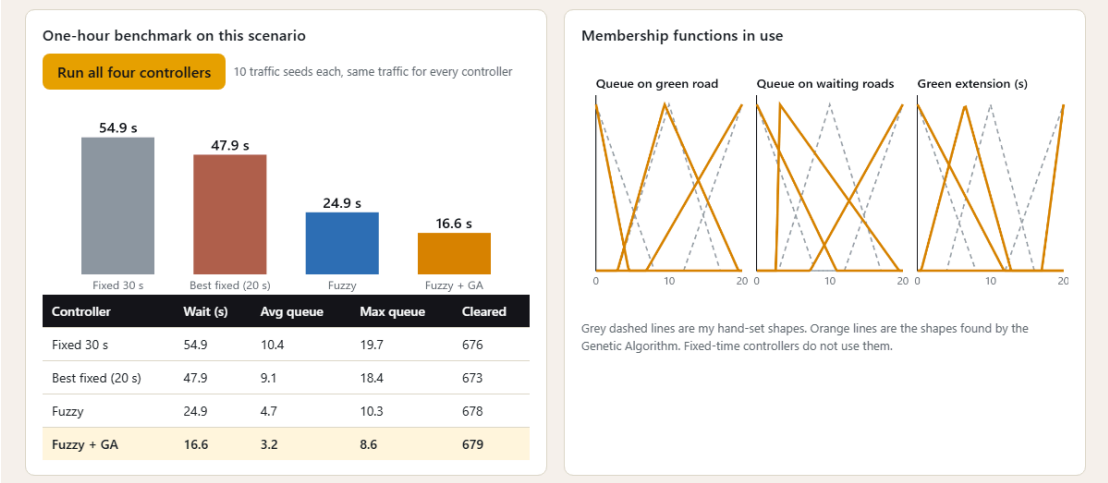


Figure 5.5: *Prototype benchmark panel and the membership functions in use*

The port is a line-by-line translation of the Python simulator and fuzzy engine. The lookup tables it builds are identical to the Python ones to three decimal places. Only the random number generator differs, so the browser's traffic is statistically the same but not vehicle-for-vehicle the same. Section 7.8 compares the two.

6. EXPERIMENTAL SETUP

6.1 Evaluation Criteria

Table 6.1: *Evaluation criteria and evidence*

#	Criterion	Question	Evidence used
1	Delay	How long does a vehicle wait on average?	Average waiting time (Sections 7.2, 7.3)
2	Queues	How long do the queues get?	Average and maximum queue (Section 7.3)
3	Volume	Does the junction still clear the demand?	Throughput (Section 7.3)
4	Value of tuning	What does the Genetic Algorithm add?	Hand-set vs. tuned, convergence curve (Sections 7.4 to 7.6)
5	Generalisation	Does it work on traffic it was not tuned on?	Training score vs. test score (Section 7.4)
6	Consistency	Is the result stable from run to run?	Standard deviation over 10 seeds (Section 7.7)

6.2 Simulation Parameters

Table 6.2: *Simulator parameters*

Parameter	Value
Run length	3600 s (one hour of traffic), one-second steps
Discharge rate	1 vehicle per 2 s of green
Lost time per phase change	3 s
Fuzzy green limits	Minimum 6 s, maximum 60 s
Switch rule	Change phase if the computed extension is under 4 s
Input and output scale	0 to 20 (longer queues are read as 20)

6.3 Traffic Scenarios

Table 6.3: *Traffic scenarios*

Scenario	Arrival rate per road (vehicles per hour)	Total demand per hour	Purpose
Light	144 on every road	576	Fixed timers waste green on empty roads
Heavy	324 on every road	1,296	Junction close to its capacity
Unbalanced	360 on North, 108 on the other three	684	One busy road; equal greens suit nobody

6.4 Controllers Compared

Table 6.4: *Controllers compared*

Controller	How it decides green time	Role
Fixed 30 s	30 s of green for every road in every cycle	Baseline
Best fixed (20 s)	The green time with the lowest mean wait in a sweep from 5 s to 60 s	Stronger baseline
Fuzzy (hand-set)	Nine rules with the membership functions of Figure 5.2	Proposed, stage 1
Fuzzy + GA	Same rules, membership functions tuned by the Genetic Algorithm	Proposed, stage 2

The second baseline matters. Comparing only against a 30 second timer would flatter the fuzzy controller, because 30 seconds may simply be a poor choice for this junction. The sweep gives the fixed timer its best chance.

6.5 Training and Test Protocol

Seeds 1, 2 and 3 are used for training. With three scenarios this gives nine one-hour traffic sets, and both the fixed-green sweep and the Genetic Algorithm see only these nine. Seeds 101 to 110 are used for testing. Every number in Chapter 7, unless marked as a training score, is the mean over these ten unseen runs of one scenario. Table 6.5 counts the simulation runs in the study.

Table 6.5: *Simulation runs in the study*

Stage	Calculation	One-hour runs
Fixed-green sweep	12 green times x 9 training sets	108
Genetic Algorithm	41 evaluations of the population x 30 candidates x 9 training sets	11,070
Final test	3 scenarios x 10 seeds x 4 controllers	120
Queue traces	1 unbalanced run x 4 controllers	4
Total		**11,302**

7. RESULTS AND ANALYSIS

7.1 Choosing the Fixed-Time Baseline

Figure 7.1 shows the mean wait on the training traffic for each fixed green time. The best value is 20 seconds (44.3 s of waiting). The usual 30 seconds gives 53.1 s, and waiting grows steadily beyond that because every road waits longer for its turn. Very short greens are worse still. With 5 seconds of green only two vehicles can leave per turn while almost three arrive per cycle in heavy traffic, so the queues never clear and the mean wait is 351 s (left off the chart). The 10 and 15 second timers sit close to that limit in heavy traffic, which is why their bars are high and not in order.

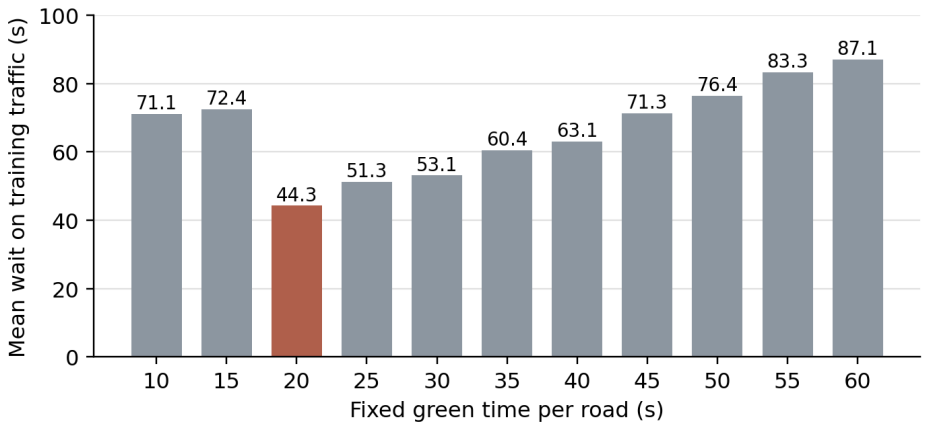


Figure 7.1: *Mean wait for each fixed green time on the training traffic (best value highlighted)*

7.2 Waiting Time

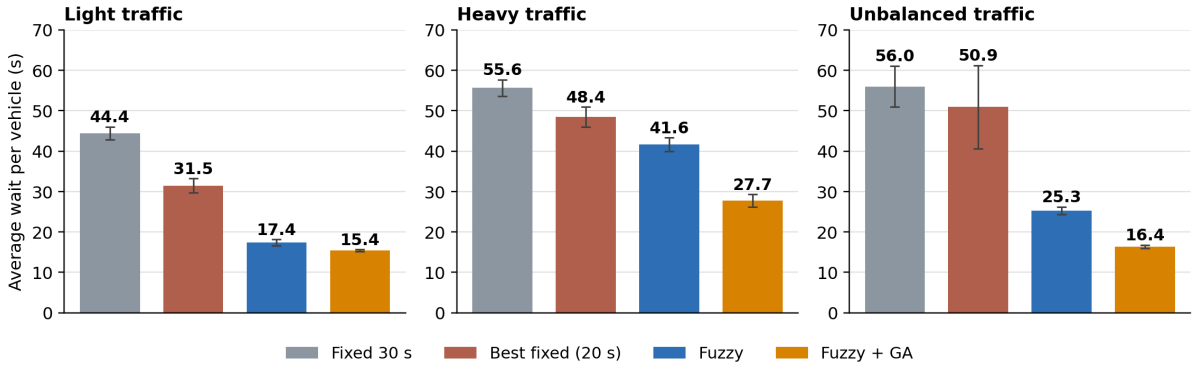


Figure 7.2: *Average waiting time per vehicle on unseen traffic (bars show the mean of 10 runs, whiskers one standard deviation)*

The tuned fuzzy controller has the lowest waiting time in every scenario (Figure 7.2). Against the 30 second timer the reduction is 65% in light traffic, 50% in heavy traffic and 71% in unbalanced traffic. Against the best fixed timer it is still 51%, 43% and 68%. The hand-set fuzzy controller is already better than both fixed timers everywhere, but by very different margins: 61% under the 30 second timer in light traffic and only 25% in heavy traffic.

The ordering of the scenarios makes sense. In light traffic a fixed timer spends most of each green on an empty road, and a controller that switches as soon as the queue is served removes that waste. In unbalanced traffic the fixed timer gives the busy road the same share as the quiet roads, which is the one situation it handles worst. In heavy traffic every road has vehicles almost all the time, so there is less wasted green to recover, and the gain is smallest.

7.3 Full Results

Table 7.1: *Results on unseen traffic (mean of 10 one-hour runs per scenario)*

Scenario	Controller	Wait (s)	Avg queue	Max queue	Throughput
Light	Fixed 30 s	44.4	7.2	10.5	571.8
	Best fixed (20 s)	31.5	5.1	8.8	576.8
	Fuzzy (hand-set)	17.4	2.8	6.5	578.4
	Fuzzy + GA	**15.4**	**2.5**	**6.0**	**578.8**
Heavy	Fixed 30 s	55.6	20.2	20.8	1291.1
	Best fixed (20 s)	48.4	17.6	19.9	1294.1
	Fuzzy (hand-set)	41.6	15.2	14.0	1294.3
	Fuzzy + GA	**27.7**	**10.1**	**12.8**	**1300.2**
Unbalanced	Fixed 30 s	56.0	10.8	21.0	684.3
	Best fixed (20 s)	50.9	9.8	20.1	682.3
	Fuzzy (hand-set)	25.3	4.9	10.2	686.6
	Fuzzy + GA	**16.4**	**3.2**	**8.3**	**688.4**

Queues follow the waiting times. In unbalanced traffic the longest queue on any road falls from 21.0 vehicles with the 30 second timer to 8.3 with the tuned controller, which on a real road is the difference between a queue that blocks the previous junction and one that does not.

Throughput is almost the same for all four controllers, within 1% in every scenario, and close to the demand in Table 6.3. This is expected and it is an honest limit on the claim being made. Demand in all three scenarios is below the capacity of the junction, so every controller eventually clears the vehicles that arrive. The controllers differ in how long those vehicles wait, not in how many get through.

Table 7.2: *Reduction in average waiting time*

Comparison	Light	Heavy	Unbalanced
Fuzzy (hand-set) vs. Fixed 30 s	61%	25%	55%
Fuzzy (hand-set) vs. Best fixed	45%	14%	50%
Fuzzy + GA vs. Fixed 30 s	65%	50%	71%
Fuzzy + GA vs. Best fixed	51%	43%	68%
Fuzzy + GA vs. Fuzzy (hand-set)	11%	33%	35%

7.4 Convergence of the Genetic Algorithm

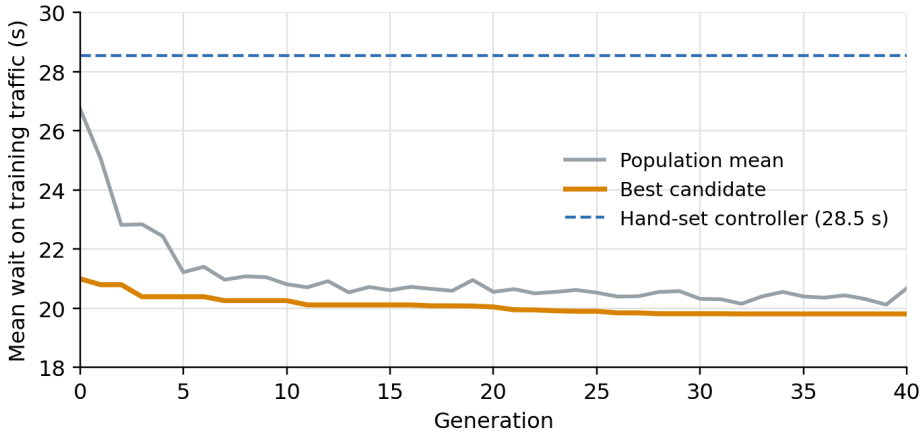


Figure 7.3: *Best and mean fitness of the population over 40 generations*

The hand-set controller scores 28.5 s on the training traffic. The best candidate in the very first population, which is only the hand-set genes with random noise added, already scores 21.0 s (Figure 7.3). Most of the gain therefore came from the first random step away from the hand-set shapes and not from forty generations of careful evolution. This was not what was expected, and it says more about the starting point than about the algorithm: the symmetric, evenly spread shapes that look reasonable on paper were far from good for this junction. The

Genetic Algorithm then refined the best candidate from 21.0 s to 19.8 s, with the last noticeable improvement around generation 30.

The training score of the tuned controller is 19.8 s. Its mean score over the three scenarios on the ten unseen seeds is also 19.8 s. The tuned controller performs the same on traffic it has never seen, so it has learned something about the junction and not memorised nine particular hours.

7.5 What the Optimizer Changed

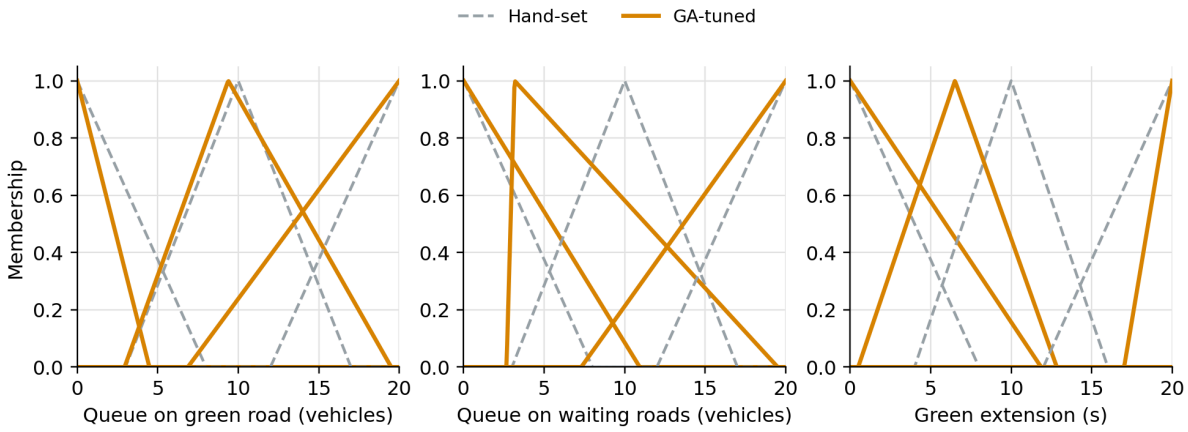


Figure 7.4: *Membership functions before (dashed grey) and after (solid orange) tuning*

Table 7.3: *Hand-set and tuned membership parameters*

Variable	Version	short_end	medium_left	medium_peak	medium_right	long_start
Green road queue	Hand-set	8	3	10	17	12
	Tuned	4.48	2.94	9.38	19.50	6.87
Waiting queue	Hand-set	8	3	10	17	12
	Tuned	10.93	2.66	3.16	19.50	7.30
Green extension	Hand-set	8	4	10	16	12
	Tuned	11.86	0.50	6.51	12.83	17.01

Three changes stand out in Figure 7.4 and Table 7.3:

- **On the green road, Long starts sooner.** A queue begins to count as Long from about 7 vehicles instead of 12, and Short ends at about 4.5 instead of 8. A busy road is

recognised as busy earlier.

- **On the waiting roads, Medium peaks much earlier.** Its peak moved from 10 vehicles to about 3. The controller becomes aware of waiting traffic almost as soon as there is any.
- **The extensions are shorter.** The Medium extension now peaks at 6.5 s instead of 10 s, and a Long extension is reserved for the top of the scale.

7.6 Decision Surfaces

The membership functions only matter through the decisions they produce, so the two lookup tables were compared directly (Figure 7.5). The black line marks the 4 second threshold: below it the controller switches, above it the controller extends.

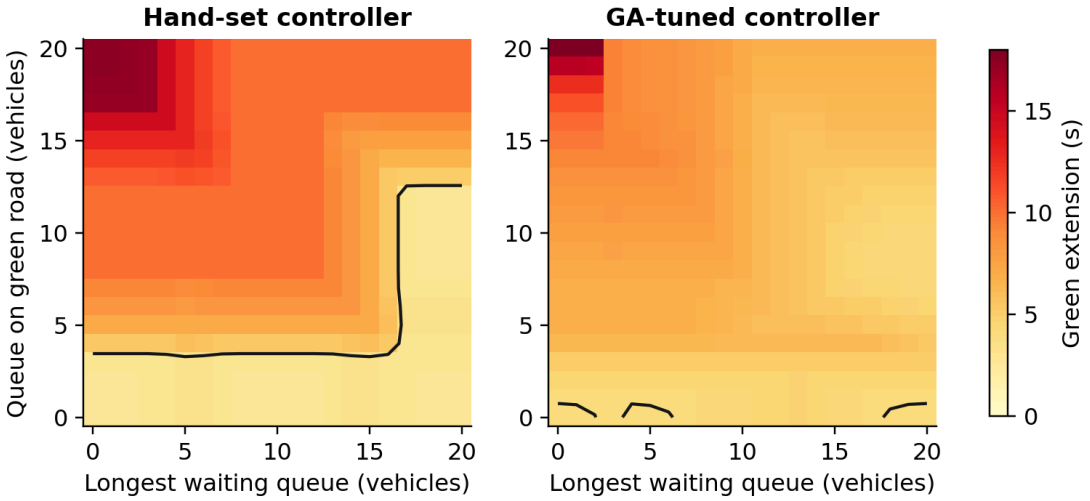


Figure 7.5: *Green extension for every pair of inputs; the black line is the 4 s switch threshold*

The hand-set controller extends in 321 of the 441 input combinations. The tuned controller extends in 432. At the same time its extensions are shorter over most of the table (Table 7.4). In plain words, the tuned controller **asks for less time but asks far more often**. With a queue of 3 on the green road and 10 waiting elsewhere, the hand-set controller computes 2.9 s and switches, leaving three vehicles behind and paying 3 seconds of lost time. The tuned controller computes 5.0 s, clears two more vehicles and then decides again.

Table 7.4: *Extension in seconds for sample inputs*

Green road queue	Waiting queue	Hand-set	Tuned	Effect of tuning
2	0	2.7 (switch)	4.4	Stays to clear a short queue
3	10	2.9 (switch)	5.0	Stays to clear a short queue
10	2	10.0	7.8	Shorter step, earlier re-check
10	10	10.0	6.9	Shorter step, earlier re-check
15	15	7.9	5.8	Shorter step, earlier re-check
20	0	17.4	19.1	Longer when nobody else is waiting

This explains why tuning helped most in heavy and unbalanced traffic. Every phase change costs three seconds in which nobody moves. A controller that leaves a road while vehicles are still queued has to come back for them a full cycle later and pays the lost time again. Serving the queue in short, repeated extensions wastes less green and switches at better moments.

7.7 Behaviour Over Time and Consistency

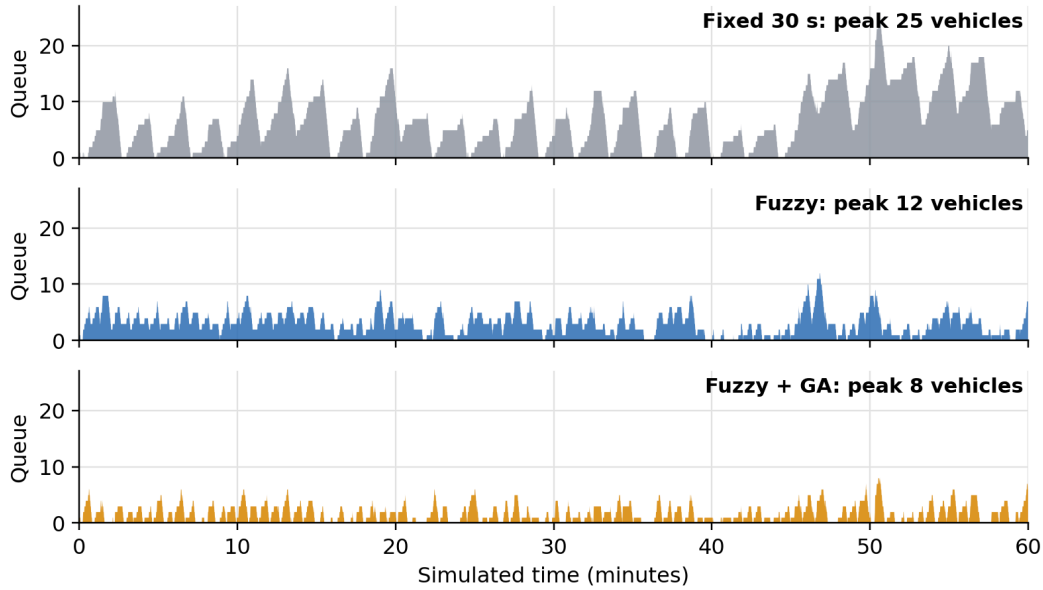


Figure 7.6: *Queue on the busy road during one unseen hour of unbalanced traffic*

Figure 7.6 follows the queue on the busy North road through one unseen hour of the unbalanced scenario. Under the 30 second timer the queue repeatedly builds into spikes and reaches 25 vehicles at its worst. With the hand-set fuzzy controller the peak is 12, and with the tuned controller it is 8, and the queue returns to zero many times during the hour.

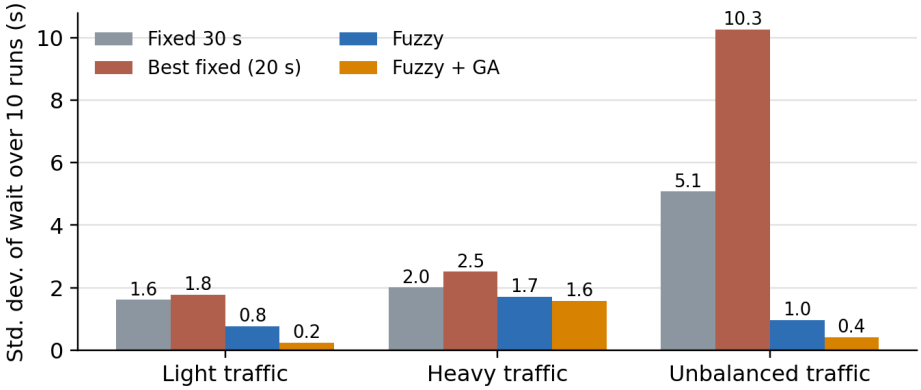


Figure 7.7: *Run-to-run spread of the average waiting time over the 10 test seeds*

The tuned controller is also the most consistent (Figure 7.7). In unbalanced traffic the standard deviation of its waiting time over the ten runs is 0.4 s, against 5.1 s for the 30 second timer and 10.3 s for the 20 second timer. The 20 second timer has the lowest fixed-time mean but the largest spread, because its green is only just enough for the busy road: in an hour with slightly more arrivals than average its queue runs away. A fuzzy controller absorbs such an hour by holding the green a little longer.

7.8 Cross-Check with the Browser Prototype

The JavaScript port was run on the same scenarios with ten seeds of its own random generator (Table 7.5). The two implementations agree within about 3 s in every cell and they rank the four controllers in the same order in every scenario. Since the two programs share no code and use different random traffic, this agreement is good evidence that the results come from the model and not from a mistake in one implementation.

Table 7.5: *Average waiting time in seconds, Python experiment vs. browser prototype*

Controller	Light (Py / JS)	Heavy (Py / JS)	Unbalanced (Py / JS)
Fixed 30 s	44.4 / 44.1	55.6 / 55.8	56.0 / 54.9
Best fixed (20 s)	31.5 / 31.8	48.4 / 46.7	50.9 / 47.9
Fuzzy (hand-set)	17.4 / 17.2	41.6 / 40.1	25.3 / 24.9
Fuzzy + GA	15.4 / 15.2	27.7 / 26.9	16.4 / 16.6

7.9 Summary Against Objectives

Table 7.6: *Results summarised against the objectives*

Objective	Verdict	Key evidence
1. Simulate the intersection	Met	Seeded Poisson arrivals, discharge and lost time; 11,302 one-hour runs
2. Design the fuzzy controller	Met	Two inputs, one output, nine rules, centroid; worked example reproduces the table value
3. Optimize with a Genetic Algorithm	Met	Training fitness 28.5 s to 19.8 s; same score on unseen traffic
4. Compare the controllers	Met	Tuned controller best in every scenario, also against the best fixed timer
5. Test under three kinds of traffic	Met	Largest gain in unbalanced traffic (71%), smallest in heavy (50%)

8. DISCUSSION

8.1 Strengths

- **Fair comparison.** Every controller runs in the same loop on the same pre-generated traffic, so differences in the results come from the controllers alone.
- **Honest baselines.** The fuzzy controller is compared against the best fixed timer found by a sweep, not only against an arbitrary 30 seconds.
- **Separate training and test traffic.** The reported numbers are from seeds the optimizer never saw, and the training and test scores agree.
- **Explained, not only measured.** The decision surfaces show what the optimizer changed and why that reduces waiting.
- **Readable controller.** The final controller is nine rules and fifteen numbers. A traffic engineer can inspect all of it.
- **Independent check.** A second implementation in another language gives the same ranking and nearly the same values.

8.2 Limitations

- **Simulated traffic.** Arrivals are Poisson with constant rates within the hour. Real traffic arrives in platoons released by upstream signals and its rate drifts during the hour.
- **Idealised sensing.** The controller is given exact queue lengths. A real detector or camera gives noisy counts, and queues above 20 are all read as 20.
- **Simple junction.** One lane per road, no turning movements, no pedestrians, no emergency vehicles.
- **Fixed phase order.** The controller cannot skip an empty road. Each empty road still costs the minimum green of 6 s plus 3 s of lost time.
- **Equal-split baselines.** Both fixed timers give every road the same green. A Webster-style plan with a longer green for the busy road would be a tougher opponent in the unbalanced scenario.
- **Rules not optimized.** Only the membership functions were tuned. The rule table and the thresholds (4 s, 6 s, 60 s) were set by hand.

- **One optimizer run.** The Genetic Algorithm was run once with one random seed. Repeating it would show how much the tuned genes vary from run to run.

8.3 Educational and Societal Impact

Within its model, the project shows that a controller small enough to explain on one slide can halve the time vehicles spend waiting, without widening a road or installing a city-wide system. The requirement on the street would be a queue estimate on each approach, which a single camera per approach can provide. The readable form of the controller is an advantage for a public system, because the people responsible for the junction can see why the signal behaved as it did.

For a student the project is a compact example of how the pieces of the course fit together: an agent that senses and acts, a fuzzy rule base that carries human judgement, and an optimizer that corrects the part of that judgement which was only a guess.

8.4 Lessons Learned

- A reasonable-looking design can be far from a good one. The hand-set membership functions were symmetric and tidy, and a random nudge improved them by a quarter.
- The cost of a decision matters as much as the decision. Most of the improvement came from avoiding unnecessary phase changes, an effect that is invisible in the rule table and obvious in the decision surface.
- Speed is a design requirement for anything inside an optimization loop. The lookup table turned an experiment that would not have finished into one that runs in about a minute.
- A baseline should be given its best chance. Sweeping the fixed green time removed about a fifth of the apparent advantage of the fuzzy controller, and what remained is more convincing for it.
- Averages hide risk. The 20 second timer had the best fixed-time mean and the worst spread.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The project set out to build a traffic signal controller that judges queues the way a person does and to tune it automatically. All five objectives were met. A Mamdani fuzzy controller with nine rules beat fixed timing in every scenario even with hand-set membership functions, cutting average waiting by 25% to 61% against a 30 second timer. Tuning the membership functions with a Genetic Algorithm removed a further 11% to 35%, for a total reduction of 50% to 71%, and the tuned controller did as well on unseen traffic as on the traffic it was trained on. The largest gain appeared where traffic was uneven, which is exactly the case a fixed timer cannot handle.

The study also showed where the gain comes from. The optimizer did not find exotic shapes. It taught the controller to serve a queue in short, repeated extensions and to leave a road only when it is nearly empty, which avoids wasting the lost time of a phase change. These conclusions hold for the simulated junction described in this report. Whether they carry over to a real junction is the first question for future work.

9.2 Future Scope

Table 9.1: *Prioritised roadmap*

Priority	Enhancement	Benefit
1	Validate on SUMO [16] with real junction counts	Tests the controller on realistic arrivals and vehicle dynamics
2	Let the controller skip empty roads and choose the next phase	Removes the minimum green and lost time spent on empty roads
3	Add a Webster-style unequal fixed plan as a baseline	A fairer opponent in the unbalanced scenario
4	Tune the rule table and thresholds as well as the membership functions	Searches the part of the design that is still hand-set
5	Compare the Genetic Algorithm with Particle Swarm Optimization [15] on the same task	Shows whether the choice of optimizer matters
6	Add noise to the queue readings	Measures robustness to imperfect sensors
7	Add emergency vehicle priority as an extra fuzzy input	Addresses a real safety need
8	Extend to a corridor of connected junctions	Moves from an isolated junction towards a network

REFERENCES

[1] F. V. Webster, “Traffic signal settings,” Road Research Technical Paper No. 39, HMSO, London, 1958.

[2] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.

[3] C. P. Pappis and E. H. Mamdani, “A fuzzy logic controller for a traffic junction,” *IEEE Trans. Systems, Man, and Cybernetics*, vol. 7, no. 10, pp. 707–717, 1977.

[4] P. B. Hunt, D. I. Robertson, R. D. Bretherton, and R. I. Winton, “SCOOT: A traffic responsive method of coordinating signals,” TRRL Laboratory Report 1014, 1981.

[5] C. L. Karr, “Genetic algorithms for fuzzy controllers,” *AI Expert*, vol. 6, no. 2, pp. 26–33, 1991.

[6] M. B. Trabia, M. S. Kaseko, and M. Ande, “A two-stage fuzzy logic controller for traffic signals,” *Transportation Research Part C*, vol. 7, no. 6, pp. 353–367, 1999.

[7] J. Niittymaki and M. Pursula, “Signal control using fuzzy logic,” *Fuzzy Sets and Systems*, vol. 116, no. 1, pp. 11–22, 2000.

[8] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI: University of Michigan Press, 1975.

[9] E. H. Mamdani and S. Assilian, “An experiment in linguistic synthesis with a fuzzy logic controller,” *Int. J. Man-Machine Studies*, vol. 7, no. 1, pp. 1–13, 1975.

[10] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA: Addison-Wesley, 1989.

[11] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2020.

[12] T. J. Ross, *Fuzzy Logic with Engineering Applications*, 4th ed. Chichester: Wiley, 2016.

[13] A. G. Sims and K. W. Dobinson, “The Sydney coordinated adaptive traffic (SCAT) system philosophy and benefits,” *IEEE Trans. Vehicular Technology*, vol. 29, no. 2, pp. 130–137, 1980.

[14] P. Koonce et al., *Traffic Signal Timing Manual*, Report FHWA-HOP-08-024, Federal Highway Administration, 2008.

[15] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proc. IEEE Int. Conf. Neural Networks*, vol. 4, 1995, pp. 1942–1948.

[16] P. A. Lopez et al., “Microscopic traffic simulation using SUMO,” in *Proc. 21st IEEE Int. Conf. Intelligent Transportation Systems*, 2018, pp. 2575–2582.

[17] C. R. Harris et al., “Array programming with NumPy,” *Nature*, vol. 585, pp. 357–362, 2020.

[18] J. D. Hunter, “Matplotlib: A 2D graphics environment,” *Computing in Science and Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[19] M. A. Khan, “FOAI: Intelligent traffic signal control using fuzzy logic and optimization,” GitHub repository, 2026. [Online]. Available: https://github.com/amaancoderx/FOAI

APPENDIX A: FUZZY ENGINE AND CONTROLLER

The core of the fuzzy controller as used in the experiment (`traffic_fuzzy_ga.py`). The rule table, the membership functions and the precomputed Mamdani inference are shown unchanged.

```
RULES = [[0, 0, 0],      # RULES[green queue][waiting queue] -> extension
         [1, 1, 0],      # 0 Short, 1 Medium, 2 Long
         [2, 1, 1]]

def memberships(p, x):
    s_end, m_l, m_p, m_r, l_start = p
    short = np.clip((s_end - x) / s_end, 0, 1)
    medium = np.clip(np.minimum((x - m_l) / (m_p - m_l), (m_r - x) / (m_r - m_p)), 0, 1)
    long_ = np.clip((x - l_start) / (UNIVERSE - l_start), 0, 1)
    return np.stack([short, medium, long_])

def build_table(genes):
    xs = np.arange(UNIVERSE + 1, dtype=float)
    ys = np.linspace(0, UNIVERSE, 81)
    mu_g = memberships(genes[0:5], xs)
    mu_w = memberships(genes[5:10], xs)
    mu_o = memberships(genes[10:15], ys)
    n = UNIVERSE + 1
    fire = np.zeros((3, n, n))                # firing strength per output set
    for i in range(3):
        for j in range(3):
            strength = np.minimum(mu_g[i][:, None], mu_w[j][None, :])
            k = RULES[i][j]
            fire[k] = np.maximum(fire[k], strength)
    agg = np.max(np.minimum(fire[:, :, :, None], mu_o[:, None, None, :]), axis=0)
    area = agg.sum(axis=2)
    centroid = np.where(area > 1e-9, (agg * ys).sum(axis=2) / np.maximum(area, 1e-9), 0.0)
    return centroid.tolist()

class FuzzyController:
    def __init__(self, table):
        self.table = table
    def first_green(self):
        return MIN_GREEN
    def extend(self, q, phase, elapsed):
        if q[phase] == 0 or elapsed >= MAX_GREEN:
            return 0
        waiting = max(q[i] for i in range(4) if i != phase)
        e = self.table[min(q[phase], UNIVERSE)][min(waiting, UNIVERSE)]
        if e < SWITCH_BELOW:
            return 0
        return min(int(round(e)), MAX_GREEN - elapsed)
```

APPENDIX B: GENETIC ALGORITHM

The repair step and one generation of the Genetic Algorithm.

```
def repair(genes):
    out = []
    for v in range(3):
        s_end, m_l, m_p, m_r, l_start = [min(max(g, 0.5), UNIVERSE - 0.5)
                                         for g in genes[v * 5:v * 5 + 5]]
        m_l, m_p, m_r = sorted([m_l, m_p, m_r])
        if m_p - m_l < 0.5: m_l = max(m_p - 0.5, 0.0)
        if m_r - m_p < 0.5: m_r = min(m_p + 0.5, float(UNIVERSE))
        out += [s_end, m_l, m_p, m_r, l_start]
    return out

nxt = [c for _, c in scored[:ELITES]]                    # elitism
while len(nxt) < POP_SIZE:
    p1 = min(rnd.sample(scored, TOURNAMENT), key=lambda s: s[0])[1]
    p2 = min(rnd.sample(scored, TOURNAMENT), key=lambda s: s[0])[1]
    child = [a if rnd.random() < 0.5 else b for a, b in zip(p1, p2)]
    child = [g + rnd.gauss(0, MUTATION_SIGMA) if rnd.random() < MUTATION_RATE else g
             for g in child]
    nxt.append(repair(child))
```